

System Implementation Final

Student Management System

Madeline Abbott and Benjamin Hogan

Table of Contents

Problem Statements and System Requirements: Page 1-3

Activity and Sequence Diagrams: Page 3-8

Implementation Planning: Page 8-9

System Architecture and Design: Page 9-11

Algorithms and Data Model Implementation: Page 11

User Interface Design and Implementation: Page 11-12

Design of Tests: Page 12

Error Handling and Debugging: Page 12

Collaboration and Code Integration: Page 12-13

Performance Optimization Plan: Page 13

Maintenance and Support Strategy: Page 13

Problem Statement

We are looking to connect students to tutors in a way that is functional and easy to understand for users who may not be used to computer systems. By allowing tutors and students to see open schedule times to sign up for they will be able to work around their own schedules and match with another student or tutor easily.

System Functionality

The Tutor Management System (TMS) will allow tutors/students to:

- Check Their Current Schedule: Any previously scheduled appointments will be seen here.
- Schedule Appointments: Set up and manage tutor/student appointments.
- Track Grades and Assignments: Students able to update their own grades to track progress.

Objectives

- Centralized Tutoring System: Provide a single platform for tutors and students to get matched together to provide students with help.
- Efficient Scheduling: Develop a more streamlined approach to appointment scheduling and management to improve efficiency.
- Grade Tracking: Enable easy tracking of medical histories and treatments.
- Data Security: Ensure that patient data is stored securely and access is controlled.

System Requirements

- User Capabilities:
 - Tutors: Access their schedule, see students looking for a tutor, and adjust their schedule if they want to add a student.
 - Student: Access their schedule, see tutors who match academic needs, and add appointments if they need help.
 - Administrators: Manage user accounts and control access levels.
- Functional Requirements:

- Frontend: A user-friendly, engaging, and appealing interface for interacting with records and scheduling.
- Backend: A reliable and capable system for handling data operations.
- Database: A relational database to store patient information, appointment schedules, and medical histories.

Typical Customers

- University Students: Any students who are struggling in a class at the college level.
- Tutors: Current or former students who want to be able to help other students succeed.

Project Planning

Software:

- Frontend: HTML, CSS, JavaScript, as well as jQuery for easier DOM manipulation.
- Backend: PHP for server-side scripting and Java for complex operations with the database.
- Database: MySQL for relational data storage.

Hardware: PCs or servers for development and deployment.

Network: Standard internet connectivity for accessing the system.

Development Approach

- Frontend: Use HTML and CSS for layout and design, and implement interactive features with JavaScript and jQuery.
- Backend: Use PHP to handle server-side logic and interactions with the database.
- Database: Design a relational schema in MySQL for managing patient records, appointments, and medical histories.
- Security: Implement user authentication and authorization in PHP, and use prepared statements in SQL to prevent SQL injection.

Development Plan

- Weeks 1-2:
 - Set up a development environment (PHP, MySQL, HTML/CSS/JS).
 - Design database schema and set up MySQL database.
- Weeks 3-4:
 - Develop the frontend interface with HTML, CSS, and JavaScript.
 - Implement basic PHP scripts for CRUD operations on patient records.
 - Create simple Java classes representing the classes in the program.
- Weeks 5-7:
 - Develop the ability to schedule appointments.
 - Add features for managing medical histories and submitting reports.
- Weeks 8-10:
 - Record midterm report.
 - Implement user authentication and authorization.
 - Ensure data security and integrity.
- Weeks 11-13:
 - Test the system for bugs and usability issues, including adding in test cases.
 - Refine and optimize features based on feedback.
- Week 14:
 - Prepare final documentation and presentation.
- Week 15:
 - Record demo and finalize project for submission.

Sequence Diagrams

Use Case 1: Updating Student Records

Actor:

User: The person who interacts with the system (ex. tutor, admin, etc.),

Object:

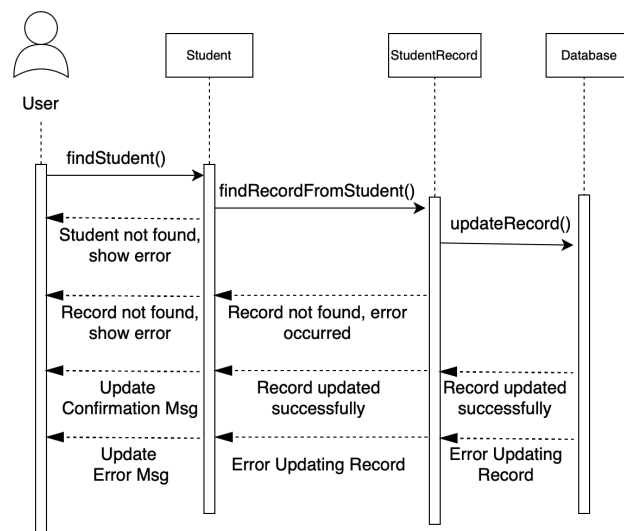
Student: the student whose records are being managed.

StudentRecord: the record of the student.

Database: the storage system for student records.

Steps of Updating a Student Record:

1. The User opens the “Manage Student Records” section in the interface.
2. The User identifies the Student object based on the input of StudentID.
3. The system sends the request to the server and finds the corresponding StudentRecord.
4. The server returns a response finding the record (either a success or error) to the interface.
5. The system processes the request and communicates with the Database by either inserting, updating, or deleting a StudentRecord.
6. The server returns a response updating the record (either a success or error) to the interface.
7. The changes are saved to StudentRecords via the Database.



Use Case 2: Scheduling an Appointment

Actor:

User: The person who interacts with the system (ex. tutor, admin, etc.),

Object:

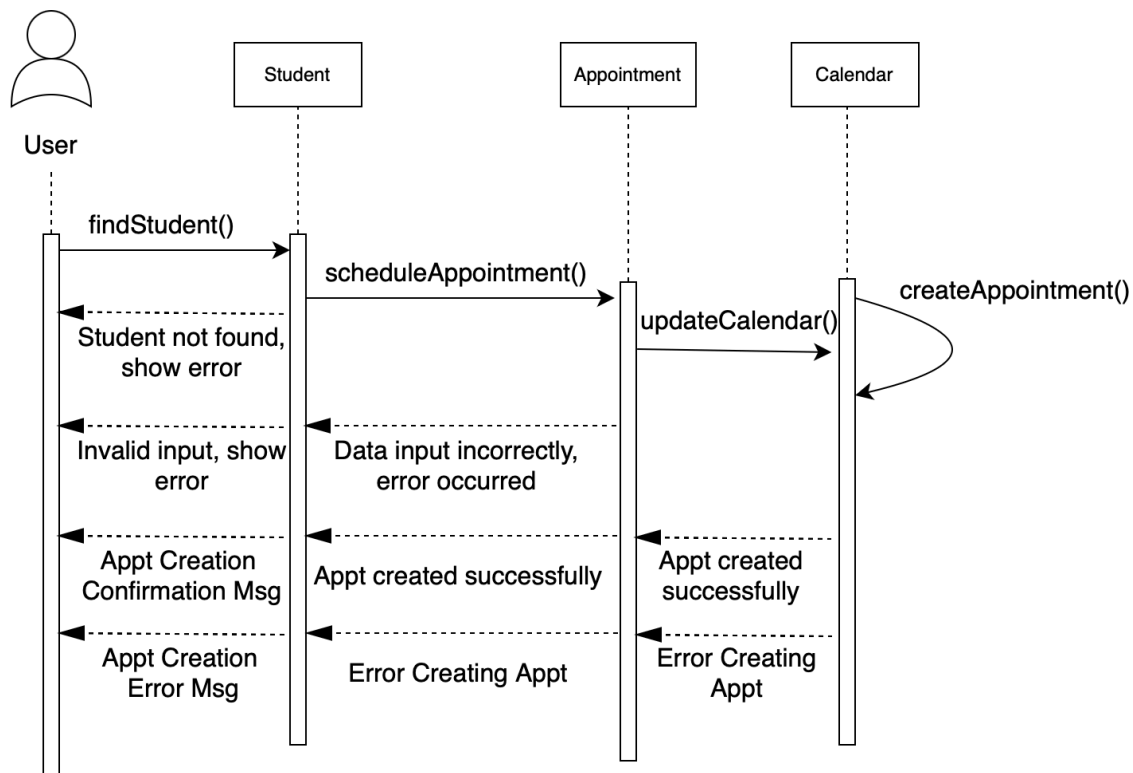
Student: The person for whom the appointment is being scheduled.

Appointment: Represents the transaction of scheduling an appointment including the time, date, and subject.

Calendar: The tool used to represent dates and times, as well as schedule appointments.

Steps of Scheduling an Appointment:

1. The User selects the Student which will have an Appointment by using their StudentID.
2. The User selects a time and date for the Appointment.
3. The system checks the Calendar to validate the time and date for the Appointment.
4. The system either returns an error or confirmation message for the Appointment.
5. The Appointment is saved in the Calendar.



Activity Diagrams

Use Case 1: Creating a Student and Student Record

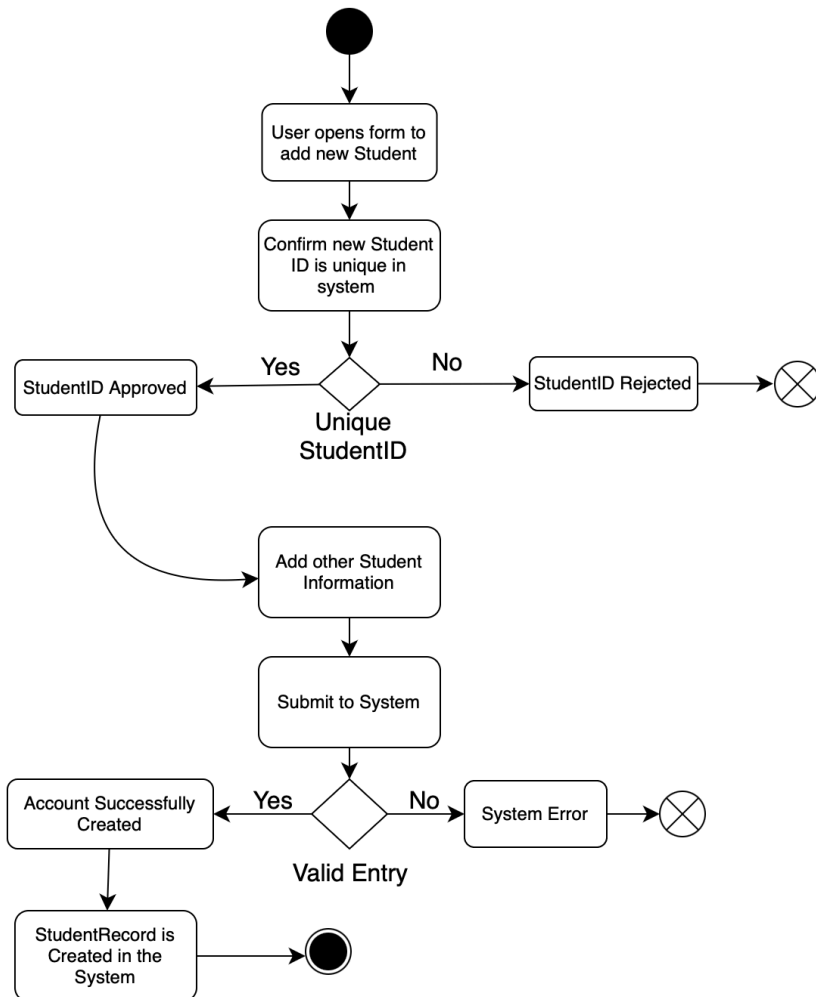
States:

Initial State: The user opens the system and begins the student creation process.

Final State: 1. The user receives confirmation of the new student creation, and a new student is added to the database. 2. The user receives an error message with an explanation, and a new student is not added to the database.

Actions:

The user is logged in and selects the “Add New Student” option. The user is given a form and inputs student information including name, student ID, date of birth, major, address, contact information, tutoring concerns, and other notes. User submits Student details. System validates the entered information. If the validation is successful, the system creates a StudentRecord. System saves the new StudentRecord to the database. System returns a success or error message to the interface accordingly.



Use Case 2: Deleting a Student and Student Record

States:

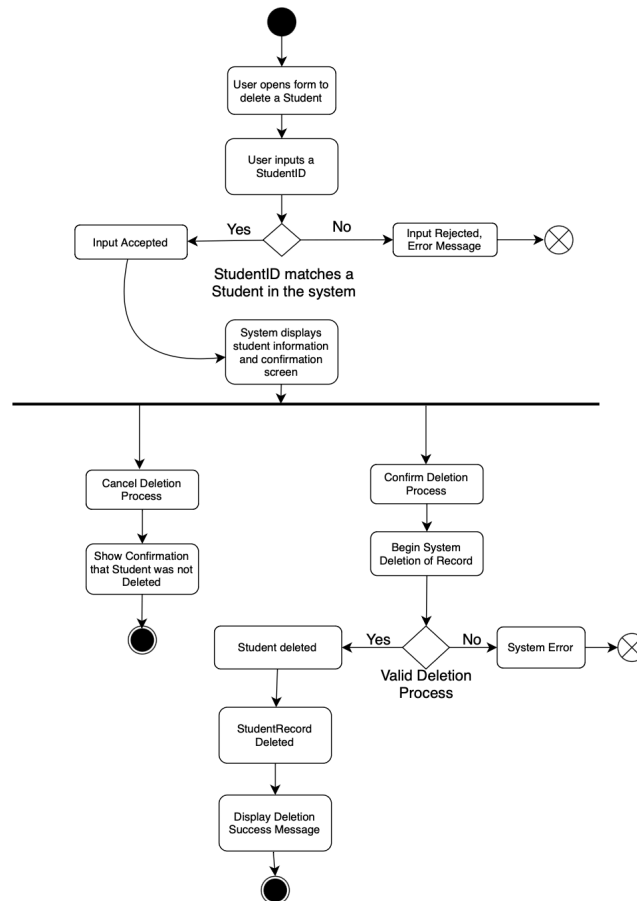
Initial State: The user opens the system and begins the student deletion process.

Final State: 1. The user receives a confirmation message, and the student and their record are successfully deleted. 2. The user receives an error message, and the student and their record remain in the system. 3. The user cancels the deletion, and the student and their record remain in the system.

Actions:

The user is logged in and selects the “Delete Student Record” option. The user is given a form to confirm the student they want to delete, where the user inputs the student ID. If both attributes do not match a student, the system returns an error. Otherwise, the system displays the student information along

with a textbox that says “Are you sure you want to delete this student’s information?” along with two buttons, each saying “yes” or “no.” If the user selects no, a confirmation message that the record was not deleted will be displayed. Otherwise, the system will delete the student and their record. If the process is successful, the system will return a confirmation message of the deletion. Otherwise, the system will return an error message that assures that the student and their record were not deleted.



Implementation Planning

To implement the student management system in a school setting, administrators would begin by setting up the required infrastructure, including a web server with PHP and SQL database support. Initial implementation would involve importing existing student and tutor data into the system’s tables and configuring tutor schedules for different appointment types. Admin users would be created first to oversee data entry and ensure accuracy during set up. The

system could then be rolled out in phases, starting with limited tutor logins to test scheduling and student management features before opening access to all tutors. Schools would also need to provide brief training for tutors and admins on how to use the interface effectively. This phased approach minimizes disruption and allows for feedback and minor adjustments before full deployment.

System Architecture and Design

Architectural Styles

Our project uses both a layered architecture and a client-server architecture.

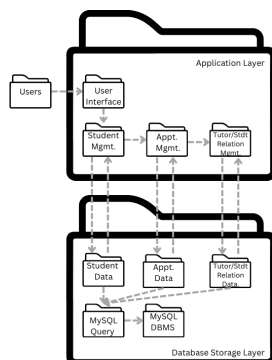
- **Layered Architecture**

- Presentation Layer: The user interface, built with HTML, CSS, and JavaScript
- Business Logic Layer: PHP handling student management operations (CRUD operations, scheduling, etc.)
- Data Access Layer: PHP interacting with the SQL database to store and retrieve data.

- **Client-server architecture**

- The client is the web browser where users interact with the system.
- The server is the machine hosting the PHP scripts and SQL database.

UML Package Diagram



Mapping Subsystems to Hardware

Our system will run on multiple computers. On the client-side, web browsers on users' machines render the HTML/CSS and run JavaScript for interactive elements. On the server-side, a web server hosts the PHP scripts and connects to the locally hosted SQL database.

Persistent Data Storage

Our system requires persistent data to store student records, appointments, and relationships with tutors. Persistent objects include things like Students, Tutors, Admins, MeetUps, TestPreps, HomeworkHelps, and TutorStudent (the relationship between tutors and students).

Network Protocol

Our system uses HTTP/HTTPS as the communication protocol. The client sends HTTP requests to the server. Then, the server processes the requests using PHP and interacts with the database before returning responses.

Global Control Flow

Execution Orders

Our system is event driven, meaning that it persistently waits for events, and each user can complete actions in a unique sequence. Users can perform actions like login, student record editing, or appointment creation. The system responds to these actions in any order, depending on user needs.

Time Dependency

Our system does not have any necessary real-time constraints because it processes actions on an event-response basis.

Concurrency

Since our system handles multiple users simultaneously, the web server and database handle concurrent requests internally. The PHP scripts don't require explicit threading since the web server manages concurrent connections.

Hardware Requirements

- **Client-Side**
 - A device with a modern web browser
 - Internet access to connect to the server
- **Server-Side**
 - A machine running a web server (ex. Apache) with PHP installed
 - A local or remote SQL database server (ex. MySQL)
 - Minimum 2 GB RAM and 10 GB disk space
 - A dual-core processor

Algorithms and Data Model Implementation

The system relies on a well structured relational database to store and manage data about students, tutors, and appointments. Each table is normalized to reduce redundancy and ensure data integrity. For example, student and tutor data are stored in separate tables with unique IDs, which are then referenced in the appointment tables. The logic for scheduling appointments involves a custom availability-checking algorithm that cross-references the required time slot with the tutor's existing schedule to prevent conflicts. These data-handling algorithms ensure accurate scheduling and reduce the likelihood of errors or double bookings. SQL queries and PHP scripts work together to manage CRUD operations while maintaining data consistency and enforcing business rules.

User Interface Design and Implementation

The user interface was designed with simplicity and clarity in mind to accommodate both tutors and admin users. The layout uses HTML and CSS for structure and styling, with intuitive navigation menus and clearly labeled buttons for tasks such as scheduling appointments, editing student profiles, and managing tutor availability. Each user role sees only the functionality relevant to their

permissions. For example, tutors are presented with options to manage their appointments and students, while admins have additional controls for adding or removing users and editing tutor data. Forms are used throughout to gather user input, and validation is implemented both client side and server side to catch errors early. Overall, the interface prioritizes both functionality and ease of use over complex design elements, ensuring that users can interact with the system efficiently with minimal training.

Design of Tests

Testing for the student management system was conducted manually and iteratively throughout development to ensure functionality and data accuracy. Key features such as appointment scheduling, user role access, and CRUD operations for students and tutors were tested using a variety of input cases. For example, the appointment scheduling logic was tested with overlapping time slots, invalid input formats, and edge cases like back-to-back bookings. Role-based access was verified by logging in as both tutor and an admin to ensure each user type could only access the features they were intended to use. Input validation and form behavior were also tested for missing or malformed data. While formal unit testing was limited, real-time debugging and scenario-based testing ensured the system functioned as expected and gracefully handled common user errors.

Error Handling and Debugging

Throughout development, error handling was built into both the front-end forms and back-end logic to create a more reliable user experience. On the client side, basic input validation helps catch missing or incorrectly formatted fields before data is submitted. On the server side, PHP error messages and conditional checks are used to catch issues like failed queries, scheduling conflicts, or unauthorized access attempts. Debugging was done using a combination of PHP's built-in error reporting, manual echo statements, and testing through the browser to follow the user flow and catch unexpected behavior in real time.

Collaboration and Code Integration

This project was a team effort, requiring consistent communication and coordination between partners. We divided responsibilities based on our strengths. If one of us did more of the coding for one section, the other would do more documentation for that section, and so on. To keep our codebase organized, we used a shared version control system. Before integration, we made sure our components followed a consistent structure and naming convention to avoid conflicts and confusion. Regular check-ins allowed us to align our progress and troubleshoot any compatibility issues early. As features came together, we worked on integrating them smoothly, making sure that database interactions, user authentication, and scheduling logic all functioned cohesively across the system.

Performance Optimization Plan

Although the system is relatively lightweight, performance was still a consideration in its design. On the front end, pages were kept simple and clean to reduce necessary load times, and server responses were structured to deliver only the necessary data. On the back end, queries were written efficiently to avoid unnecessary joins or large data pulls. In a school implementation, performance could be further optimized by enabling persistent database connections, implementing pagination for long lists of appointments or users, and expanding server capacity if the user base grows. These strategies help ensure the system remains responsible even under heavier usage.

Maintenance and Support Strategy

To support long term use of the student management system in a school setting, a clear maintenance strategy is essential. The system was built with modular, readable code to make future updates and bug fixes easier to implement. Admin users would be responsible for managing day-to-day updates, such as adding new tutors or removing graduated students. For ongoing technical support, a designated staff member or developer could be trained to handle minor issues like server errors or database inconsistencies. Future improvements, like integrated email reminders, can be added incrementally thanks to the system's scalable design. Regular backups and periodic checks of database integrity would round out a solid maintenance plan.